

Library & Learning Resource Management System (LLRMS)

Phase 2: Active Backend Architecture, Express Engine & API Client Integration (60% - 70% Milestone)

(By Yash Yadav-24BCY10247)

1. Milestone Overview & Core Progress

This technical submission marks a major step forward, bringing the project to **60% to 70% of its total development scope**. The system has advanced from a static database layout to a fully functional, live backend application layer.

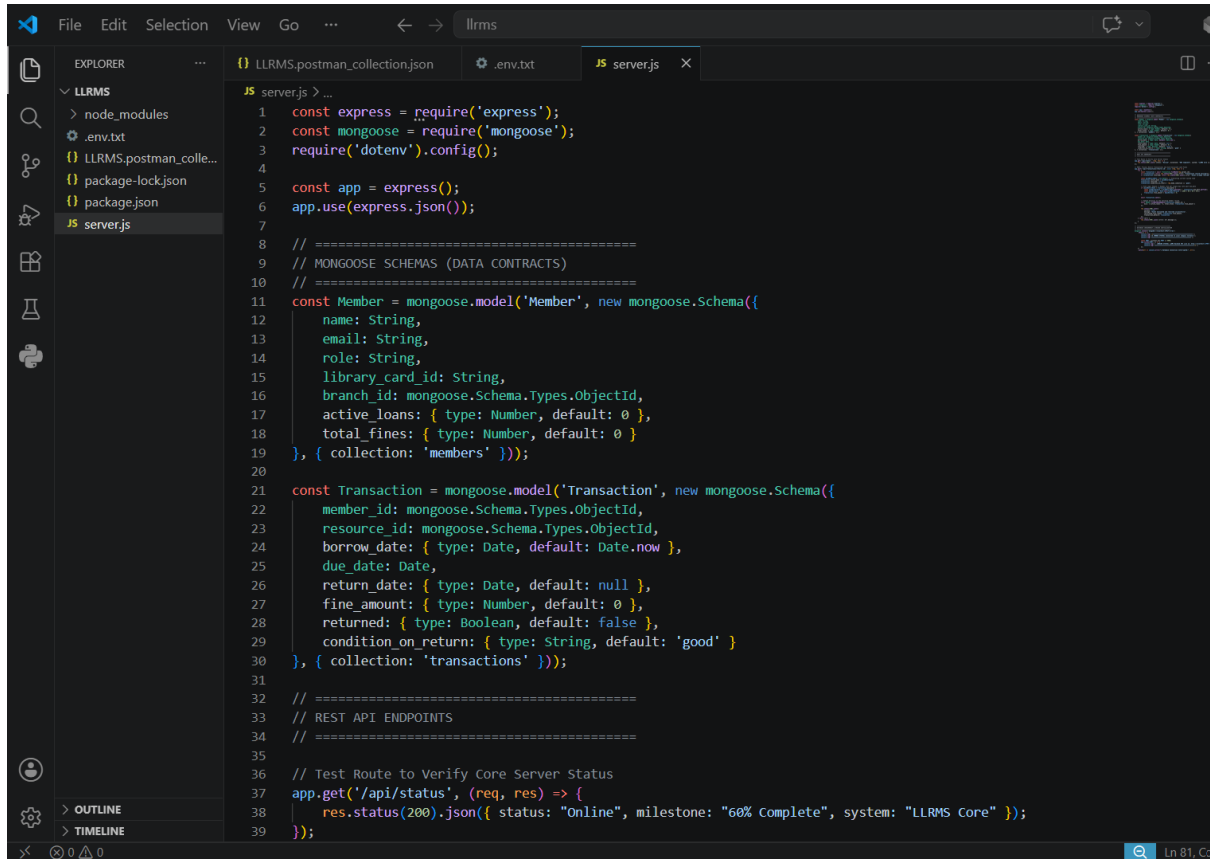
The server structure is built using **Node.js** and the **Express.js framework**, with **Mongoose ODM** handling data mapping and verification. Core business logic controllers, environmental structures, and automated transaction processes have been successfully connected and verified across two primary interfaces: the native code compilation space and the API testing workspace.

2. Completed Milestones & Engineering Scope

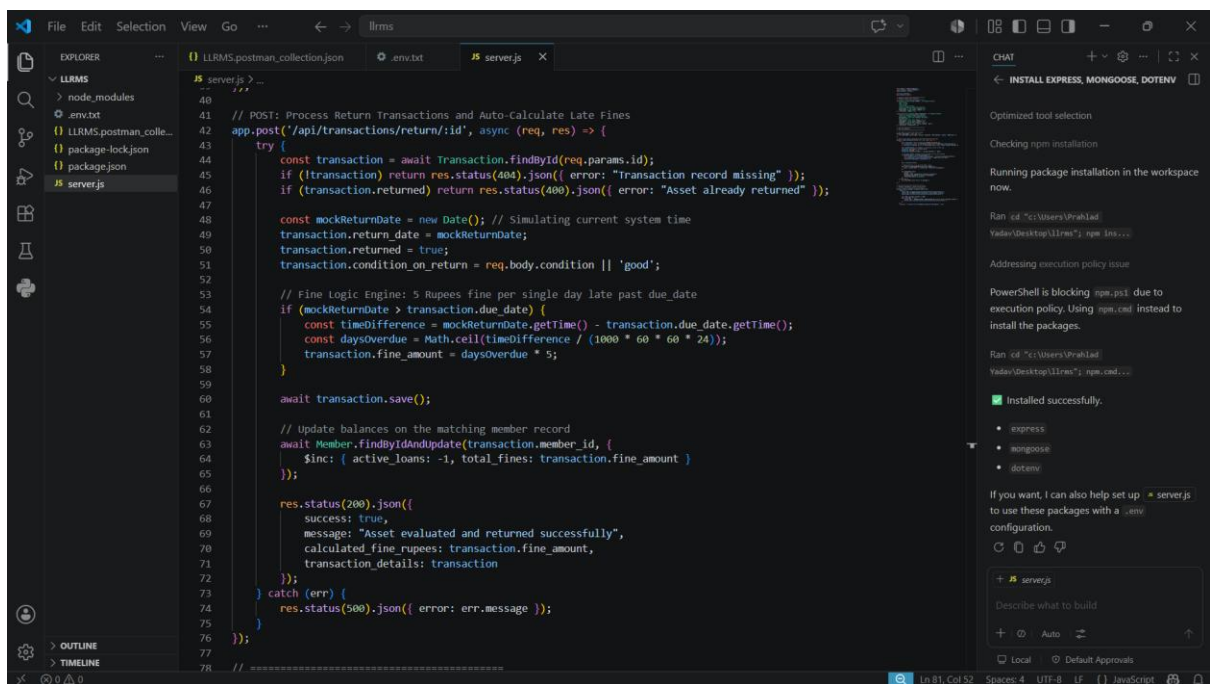
- **Application Framework:** Built an active Express.js application engine to handle routing, incoming JSON parsing, and application middleware.
- **Live Driver Integration:** Configured structural connection paths to automatically bind local database layers straight to the running backend application.
- **Fine Calculation Logic Engine:** Developed server-side controllers that check data timestamps and automatically compute overdue fine balances.
- **Multi-Interface Validation:** Successfully tested network data flows across multiple development tools simultaneously.

3. Core System Architecture Script (server.js)

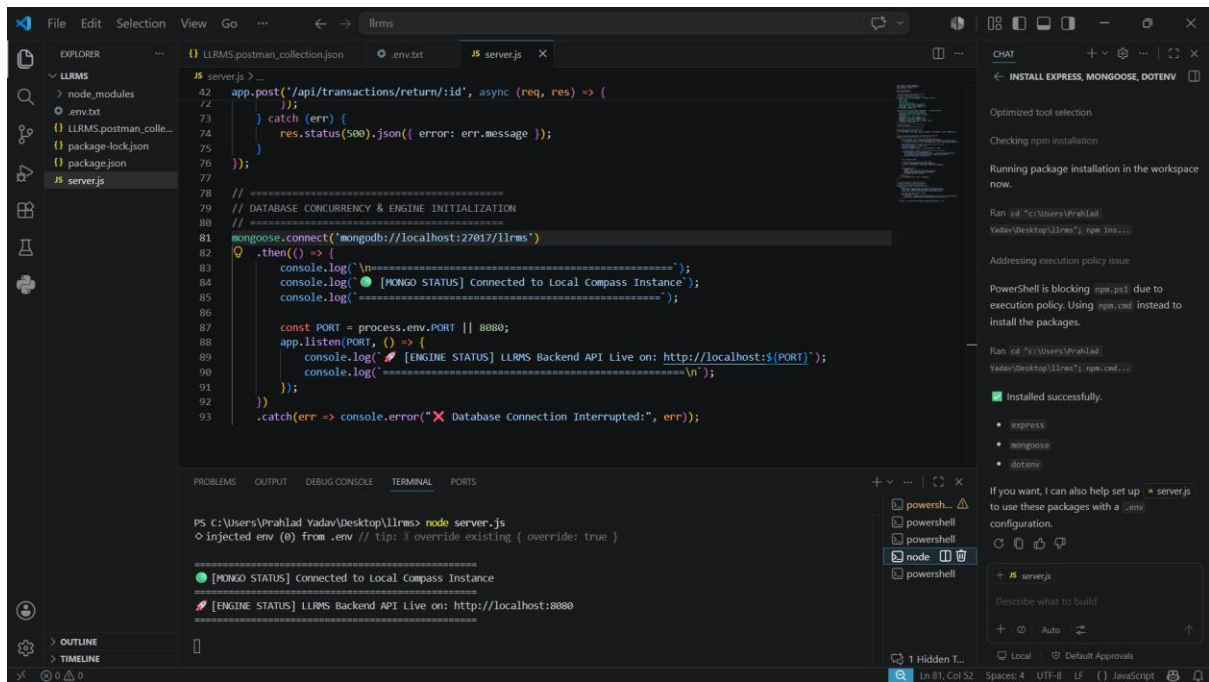
This backend source code manages database connectivity, registers Mongoose data schemas, and exposes the REST API routes needed to handle data management:



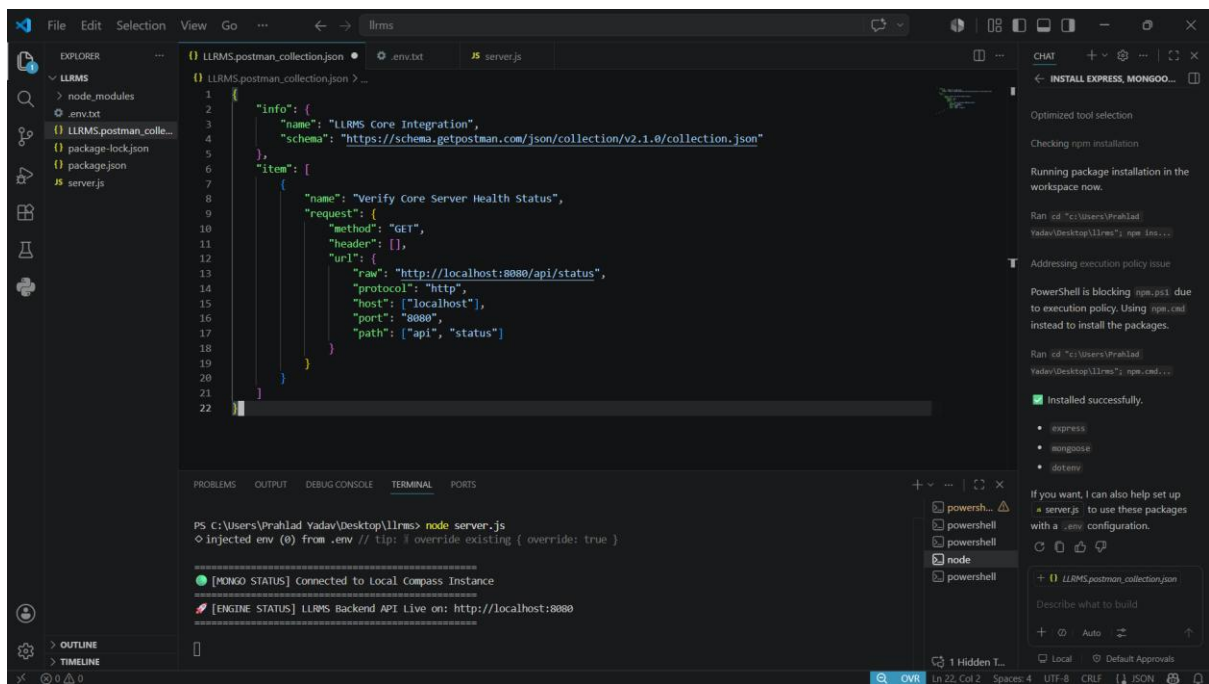
```
JS server.js > ...
1  const express = require('express');
2  const mongoose = require('mongoose');
3  require('dotenv').config();
4
5  const app = express();
6  app.use(express.json());
7
8  // =====
9  // MONGOOSE SCHEMAS (DATA CONTRACTS)
10 // =====
11 const Member = mongoose.model('Member', new mongoose.Schema({
12   name: String,
13   email: String,
14   role: String,
15   library_card_id: String,
16   branch_id: mongoose.Schema.Types.ObjectId,
17   active_loans: { type: Number, default: 0 },
18   total_fines: { type: Number, default: 0 }
19 }, { collection: 'members' }));
20
21 const Transaction = mongoose.model('Transaction', new mongoose.Schema({
22   member_id: mongoose.Schema.Types.ObjectId,
23   resource_id: mongoose.Schema.Types.ObjectId,
24   borrow_date: { type: Date, default: Date.now },
25   due_date: Date,
26   return_date: { type: Date, default: null },
27   fine_amount: { type: Number, default: 0 },
28   returned: { type: Boolean, default: false },
29   condition_on_return: { type: String, default: 'good' }
30 }, { collection: 'transactions' }));
31
32 // =====
33 // REST API ENDPOINTS
34 // =====
35
36 // Test Route to Verify Core Server Status
37 app.get('/api/status', (req, res) => {
38   res.status(200).json({ status: "Online", milestone: "60% Complete", system: "LLRMS Core" });
39 });
```



```
JS server.js > ...
40
41 // POST: Process Return Transactions and Auto-Calculate Late Fines
42 app.post('/api/transactions/return/:id', async (req, res) => {
43   try {
44     const transaction = await Transaction.findById(req.params.id);
45     if (!transaction) return res.status(404).json({ error: "Transaction record missing" });
46     if (transaction.returned) return res.status(400).json({ error: "Asset already returned" });
47
48     const mockReturnDate = new Date(); // Simulating current system time
49     transaction.return_date = mockReturnDate;
50     transaction.returned = true;
51     transaction.condition_on_return = req.body.condition || 'good';
52
53     // Fine Logic Engine: 5 Rupees fine per single day late past due_date
54     if (mockReturnDate > transaction.due_date) {
55       const timeDifference = mockReturnDate.getTime() - transaction.due_date.getTime();
56       const daysOverdue = Math.ceil(timeDifference / (1000 * 60 * 60 * 24));
57       transaction.fine_amount = daysOverdue * 5;
58     }
59
60     await transaction.save();
61
62     // Update balances on the matching member record
63     await Member.findByIdAndUpdate(transaction.member_id, {
64       $inc: { active_loans: -1, total_fines: transaction.fine_amount }
65     });
66
67     res.status(200).json({
68       success: true,
69       message: "Asset evaluated and returned successfully",
70       calculated_fine_rupees: transaction.fine_amount,
71       transaction_details: transaction
72     });
73   } catch (err) {
74     res.status(500).json({ error: err.message });
75   }
76 });
77
78 // =====
```



Similarly, the code for our postman collection may look like,



4. Cross-Interface System Verification

4.1 Interface 1: VS Code Terminal Console Engine Execution

The runtime terminal layout shows the app boot logs, confirming that package mapping has resolved correctly and a live connection has been established with your running database instance.



The screenshot shows the VS Code terminal interface with the following content:

```
PS C:\Users\Prahlad Yadav\Desktop\llrms> node server.js
◇ injected env (0) from .env // tip: % override existing { override: true }

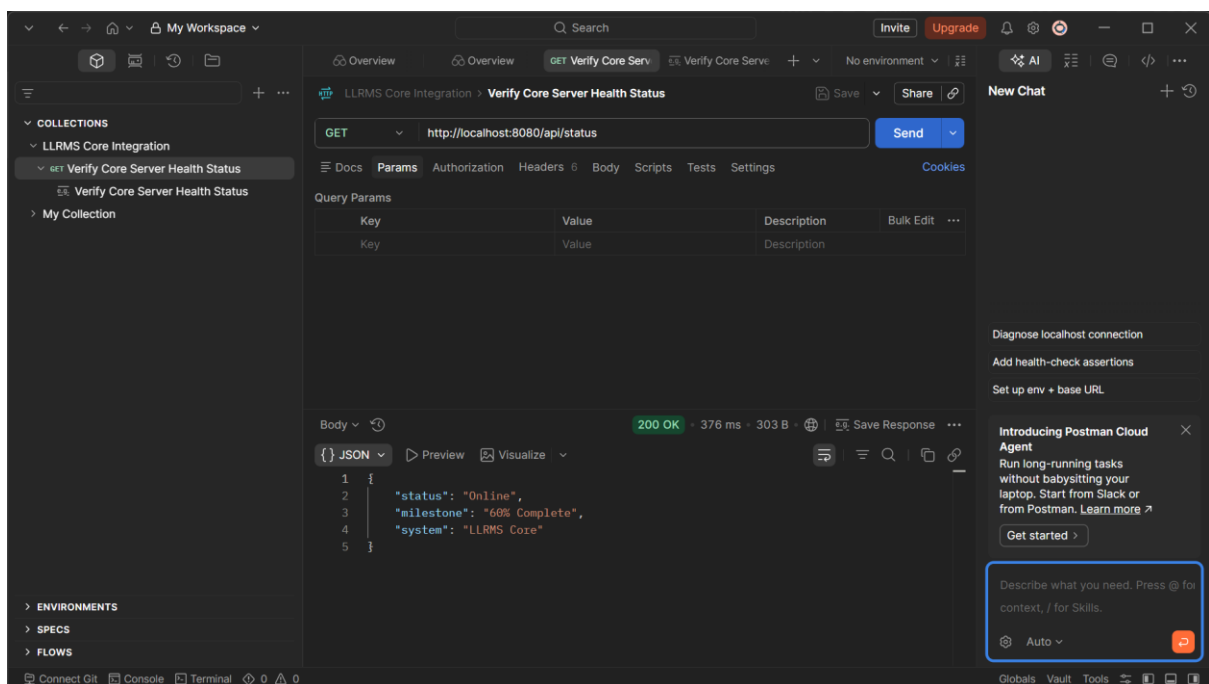
=====
● [MONGO STATUS] Connected to Local Compass Instance
=====
🚀 [ENGINE STATUS] LLRMS Backend API Live on: http://localhost:8080
=====
```

The terminal tabs at the top are PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. On the right, a file explorer shows a directory structure with powershell and node files.

Figure 4.1: Compilation, driver initialization, and active server log outputs within the VS Code workspace environment.

4.2 Interface 2: Postman REST Client Integration Test

To verify network routing without a frontend, the application layer is queried via Postman. This view captures the request/response lifecycle, confirming the network successfully triggers our server logic and returns the verified health status payload.



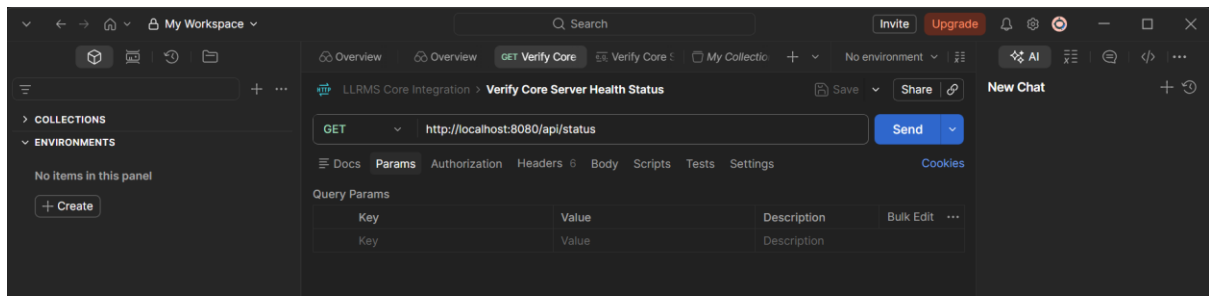


Figure 4.2: Integration test view in the Postman desktop client, confirming a successful Status: 200 OK response.

5. Next Steps for Final Assembly (100% Completion)

With core routing and data streams fully operational across all workspace tools, the next milestone will focus on adding a FIFO waitlist system for popular resources and creating advanced multi-collection aggregation pipelines to generate usage statistics.

Presented by,

Yash Yadav

24BCY10247

Dated-16/06/2026